
Rapport

**POLYTECHNIQUE
MONTREAL**

UNIVERSITÉ
D'INGÉNIERIE



Prototypage d'une application **Fractale**

ELE8307 – Prototypage rapide de systèmes numériques
Travail présenté à Jean-Pierre David et Mengyue Su

Hiver 2021

Département de génie électrique
École Polytechnique de Montréal

Dernière mise à jour : 22 avril 2021 à 16:10:23

Michael **Flandez**
Sekou **Kone**

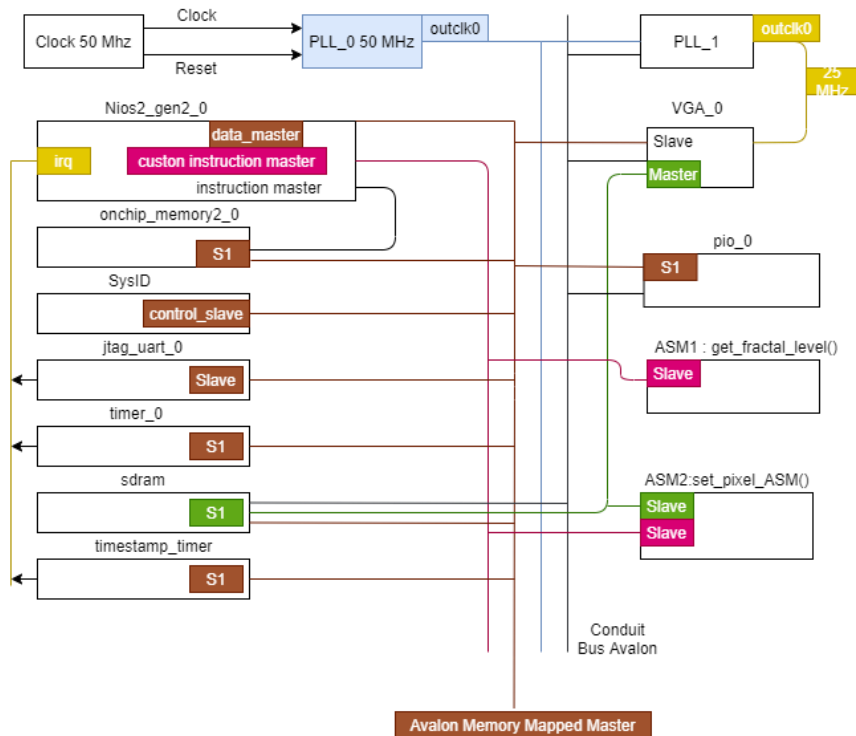
1825018
1809202

1 Introduction

Dans la vie de tous les jours, on rencontre très souvent des applications, la plupart sont déjà très bien optimisées, mais dans certains cas comme dans un contexte académique, on tombe sur des applications qui sont très peu optimales pour la plateforme pour laquelle elles ont été configurées. C'est le cas de l'application sur laquelle porte notre projet, elle a été volontairement rédigée de sorte à produire un rendement assez bas associé à un haut niveau de précision. Il a été question pour nous tout au long du projet de trouver un compromis raisonnable entre réduction du niveau de précision et augmentation des performances de l'algorithme en termes d'images par seconde. Pour arriver à notre résultat final, nous avons dû mettre en pratique les méthodes étudiées au cours de la session comme la transformation de code C en asm et la transformation d'asm en machines à état. Nous avons aussi dû faire appel aux connaissances acquises en laboratoire comme la création d'instruction spécialisée multicycle et la mise en place d'instructions utilisant le bus Avalon. Ce rapport aura donc pour objectif de retranscrire les différentes étapes par lesquelles nous sommes passés pour obtenir la version finale de notre application.

Ainsi, dans le présent rapport, nous allons commencer par faire une description générale de l'architecture et la mise en valeur de ses parties les plus ingénieuses, cela passera par la présentation des modifications apportées à la première spécification et celle des implémentations qui se retrouveront dans le code. Par la suite, nous produirons une documentation technique au projet. Celle-ci regroupera les algorithmes utilisés, les asm et leur fonctionnement ainsi que les instructions spécialisées et les périphériques Avalon développés.

2 Description générale de l'architecture et mise en valeur de ses parties les plus ingénieuses



Tous les modules sont branchés aux signaux clock et reset de PLL_0, sauf VGA qui est connecté à PLL_1
Ces signaux n'ont pas été dessinés par propriété

Figure 1 : Schéma de l'architecture générale de la partie matérielle en implémentant les deux ASM en VHDL.

On peut voir sur la figure 1, l'architecture générale de la partie matérielle ainsi que les connexions entre les différents modules par rapport aux adresses mémoires de chaque composant, par rapport aux instructions spécialisées (custom instruction master) et par rapport au conduit du bu Avalon ('wire').

Si bien on voit un fil 'outclk0' sortir du PLL_0, on le voit que ce fil n'est pas connecté. En vérité, il est branché à tous les composants sauf à VGA_0. Il rentre dans les entrées 'clock_input' et 'reset_input' qui ne sont pas affichées afin de garder la lisibilité du schéma.

- **Clock** 50 MHz: horloge maître à 50 MHz. Clock envoie l'horloge à PLL_0, et reset à tous les modules.
- **PLL_0** : horloge esclave à l'horloge Clock_50. Sa vitesse est de 50 MHz avec un petit déphasage de -3800 ps. C'est cette horloge qui donne la cadence à tous les composants du système sauf à l'écran VGA_0 qui est esclave de PLL_1. PLL_0 peut envoyer des 'resets' également.
- **PLL_1** : horloge qui tourne à autour de 25.154 MHz, esclave de l'horloge PLL_0, chargé d'envoyer l'horloge à VGA.
- **Nios2** : processeur matériel. Gère les instructions, les appels à des instructions et à des instructions spécialisées, les accès mémoires, les interruptions, les 'resets' de débogage.
- **Onchip_memory** : mémoire rapide sur le chip de 128 ko, connecté au data master et à PLL_0.
- **SysId** : périphérique permettant d'identifier le système matériel.
- **Jtag_uart_0** : lien USB.
- **Timer_0** : compteur de 32 bits de 1ms.
- **Sdram** : contrôleur. Mémoire de 64 MB où sont stockées les instructions.
- **Timestamp_timer** : deuxième compteur de 32 bits de 1us pour le profileur GNU.
- **VGA_0** : Écran périphérique, avec une adresse mémoire, connecté à la SDRAM comme maître.
- **Pio_0** : périphérique clavier d'entrée avec des boutons.
- **ASM_1** et **ASM_2** : instructions spécialisées.

On remarque qu'en plus de l'architecture de base, les deux nouvelles instructions spécialisées sont présentées. Les deux ASM sont branchées en mode esclave à 'custom instruction master' du processeur Nios2 et ils reçoivent l'horloge de 100 MHz provenant du PLL_0. L'ASM est aussi branché en mode esclave au contrôleur SDRAM, laquelle est aussi connectée au VGA qui est en fonction maître.

2.1 Modifications apportées à la première spécification

Pour la plupart des optimisations implémentées en logiciel, on n'a pas eu des modifications à faire. Mais on nommera quelques travaux subséquents à faire dans la section quatre de ce document.

Pour ce qui a à avoir avec les machines d'états, on décrira les modifications faites.

A. ASM 1 pour get_fractal_level() (Sekou Kone)

La mise en œuvre de la machine à état devant remplacer l'instruction get_fractal_level() a été faite en deux temps, d'abord, nous avons le design initial représenté à la figure suivante :

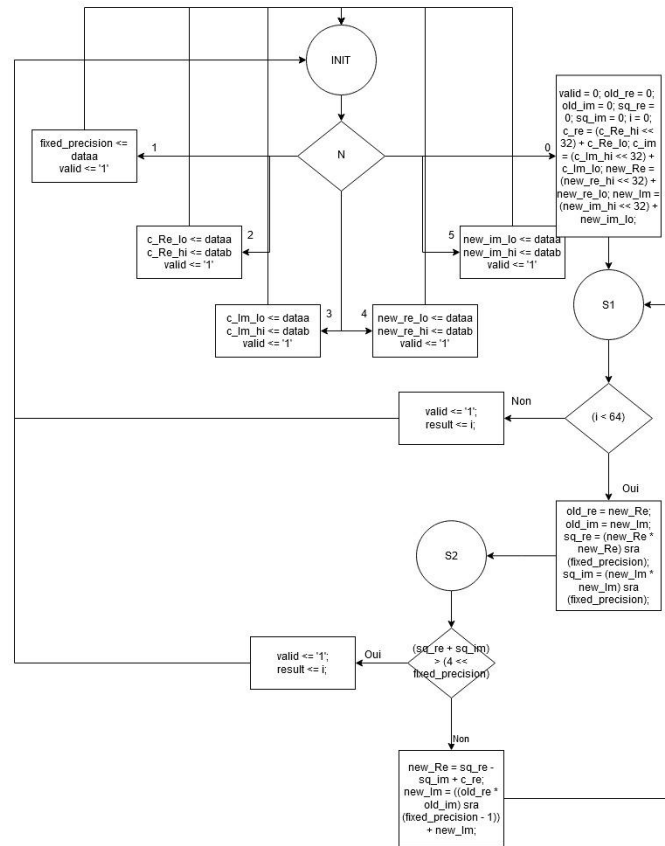


Figure 2 : ASM original pour l'instruction get_fractal_level()

Sur la base de cet asm, on peut facilement remarquer que les opérations effectuées dans chaque état sont lourdes, notamment dans les états S1 et S2 où l'on a des multiplications de deux entiers 64 bits puis des décalages. Cela nous a causé des problèmes de timing, car la fréquence d'horloge initiale était de 100 MHz ce qui était trop rapide pour effectuer toutes ces opérations en un seul cycle d'horloge. Pour pallier ce problème, nous avons d'une part diminué la fréquence de fonctionnement du système et d'autre part rajouté un état intermédiaire entre S1 et S2 pour y réaliser les décalages.

B. ASM2 pour set_pixel_ASM() (Michael Flandez)

L'implémentation de cette machine d'états a posé une difficulté dans un premier temps. La figure suivante montre le schéma de l'ASM original.

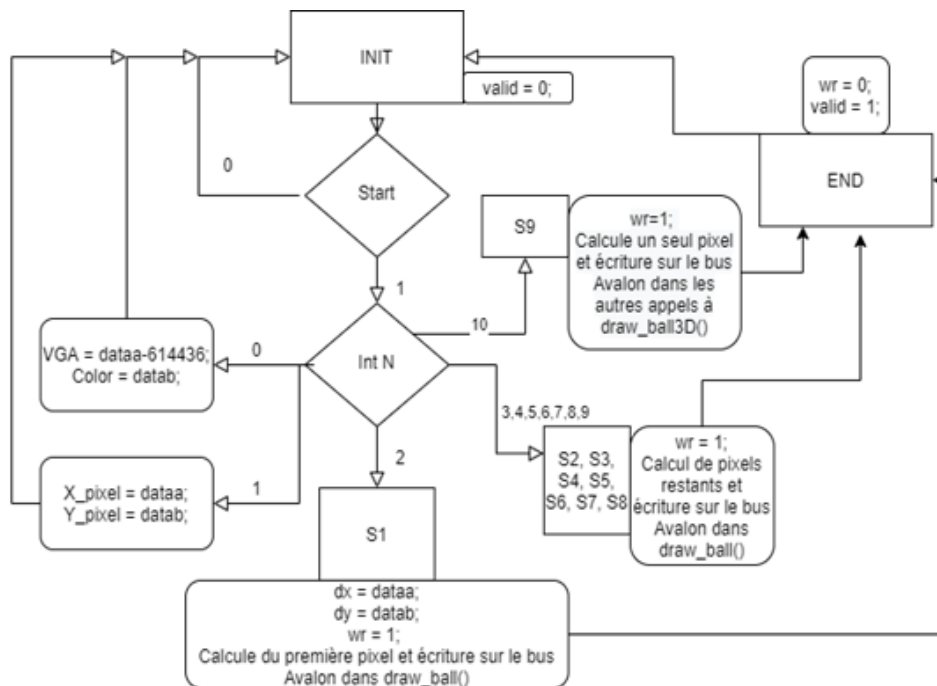


Figure 3: Schéma de l'ASM original pour l'instruction spécialisée set_pixel_ASM()

En regardant la figure 3, on voit la condition 'start' suivie par la condition 'N'. Cette condition est devenue trop lourde est donc on a dû modifier le schéma de façon à faciliter les comparaisons (**if**) en les réduisant en nombre. Dans le schéma, on a un total de dix conditions à examiner dans l'état **INIT** et bien que la simulation sur **ModelSIM** soit correcte, les résultats des instructions étaient aberrants. La solution à ce problème est de rajouter des états en allégeant chaque état. Ceci sera expliqué en plus de détails dans la section 3.2.

2.2 Implémentations originales et implémentations réutilisées

- Ici on doit faire mention à l'utilisation d'un algorithme pour calculer la racine carrée d'un **float** par interpolation (Stackoverflow, 2009).
- On s'est inspiré du cours ELE8307 – Prototypage rapide de systèmes numériques où l'on a implémenté une instruction spécialisée dessinant une couleur à l'adresse d'un pixel sur un écran VGA (adresse) donné. Cet exercice nous était donné afin de nous introduire aux concepts de l'utilisation des instructions spécialisés qui écrivent sur le bus Avalon.

3 Documentation technique du projet

3.1 Algorithmes utilisés

A. ASM 1 pour `get_fractal_level()` (Sekou Kone)

Initialement, la fonction `get_fractal_level()` s'articule autour de :

- Une boucle `for` de 64 itérations pour obtenir les différents degrés de profondeur de la fractale. Dans cette boucle `for`, on actualise tout d'abord les valeurs réelle et imaginaire puis on calcule leurs carrés,
- Un test `if` qui vérifie si on a divergence (le module du nombre complexe est supérieur à 2) si on a effectivement un module trop grand, on retourne immédiatement la valeur de l'itération, tandis que si le module est petit on calcule les prochaines valeurs réelle et imaginaire puis on retourne au début de la boucle.

C'est cet algorithme qui a été implémenté en `asm` puis en machine à état pour obtenir l'instruction spécialisée `multicycle` qui la remplace.

B. ASM2 pour `set_pixel_ASM()` (Michael Flandez)

Les fonctions originales des fichiers `VGA.h` et `VGA.cpp` calculent l'adresse d'un pixel à dessiner sur un objet pointeur `current_VGA`. Chaque fois que cette fonction est appelée, elle passe une valeur **X** entre 0 et 640 et une autre valeur **Y** entre 0 et 480 en paramètre pour ensuite calculer $X + Y * 640$. Ce calcul est allégé en faisant des décalages en matériel de la façon suivante:

$$X + Y \text{ décalé de } 9 + Y \text{ décalé de } 7. \quad (1)$$

Originellement dans la fonction `draw_ball3d()`, cette fonction est appelée huit fois dans une boucle dessinant en fait huit pixels successivement en calculant des sommes et des différences et tous les points de même couleur. Avec l'implémentation ASM de cette partie du code, on fait appel à la fonction comme elle sera décrite dans la partie 3.2 B de ce document.

3.2 ASM et leur fonctionnement

A. ASM 1 pour `get_fractal_level()` (Sekou Kone)

Dans cette section, nous décrirons de manière détaillée la manière de modifier l'`asm` original de la fonction `get_fractal_level()` pour qu'il fonctionne adéquatement.

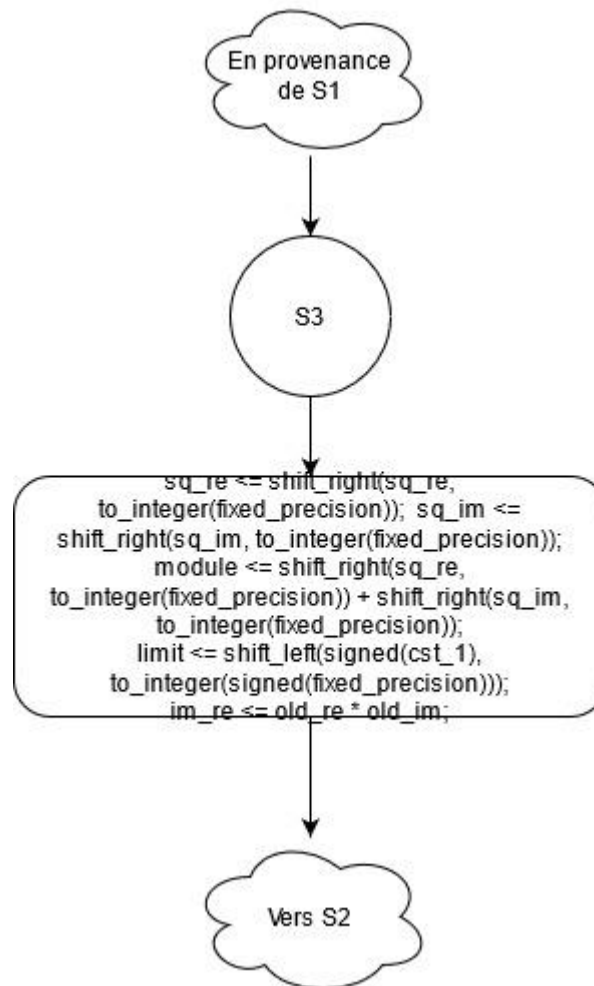


Figure 4 : Ajout d'un état intermédiaire dans `get_fractal_level()`

En observant la figure 4 et la figure 2, on peut se rendre compte qu'on a quatre états différents dans cette nouvelle version de l'asm `get_fractal_level()`. En effet, nous avons tout d'abord un état INIT dans lequel les différentes valeurs du paramètre N font la sauvegarde des variables nécessaires. Pour la valeur $N = 0$ on a le début de l'algorithme `get_fractal_level` pour lequel on se rend dans l'état S1 qui matérialise la boucle `for` de 64 itérations, ensuite on se rend inconditionnellement dans l'état S3 où on fait les décalages nécessaires qui suivent les multiplications de l'état précédent. De même, on se rends ensuite inconditionnellement dans l'état S2 qui représente le test `if` pour le retour de la valeur de l'itération.

L'ajout de cet état S3 intermédiaire était nécessaire, car il nous a permis d'augmenter notre fréquence d'horloge maximale à 80.02 MHz ce qui nous a amenés à réduire la fréquence de la pll0 de 100 MHz à 50 MHz. Cette réduction a eu comme conséquence de réduire par 2 la vitesse de rafraîchissement de notre application. Malgré cette réduction, nous avons désormais l'assurance d'avoir une application qui fonctionnait la plupart du temps.

Rapport – Fractale

Avant d'implanter la machine à état au sein de notre code éclipse, nous avons procédé à la simulation de cette dernière pour observer son comportement dans un test bench et voir si les résultats retournés étaient les bons. Cette simulation nous a permis de valider le comportement de la machine à état. Pour cette vérification, nous nous sommes basés pour les valeurs d'entrée sur celles produites à la 171 itération de l'appel de `get_fractal_level()`. Les valeurs d'entrée étaient donc :

- N = 1 : sauvegarde de `fixed_precision` à la valeur `x"0000001a"` ;
- N = 2 : sauvegarde de `c_Re_lo` et de `c_Re_hi` respectivement à `x"fce147af"` et à `x"ffffffff"`,
- N = 3 : sauvegarde de `c_Im_lo` et de `c_Im_hi` respectivement à `x"00a1cac0"` et à `x"00000000"`,
- N = 4 : sauvegarde de `new_re_lo` et de `new_re_hi` respectivement à `x"fd888889"` et à `x"ffffffff"`,
- N = 5 : sauvegarde de `new_im_lo` et de `new_im_hi` respectivement à `x"fbf5c290"` et à `x"ffffffff"`,
- N = 0 : On s'attend à recevoir en sortie une valeur de `i = 2`.

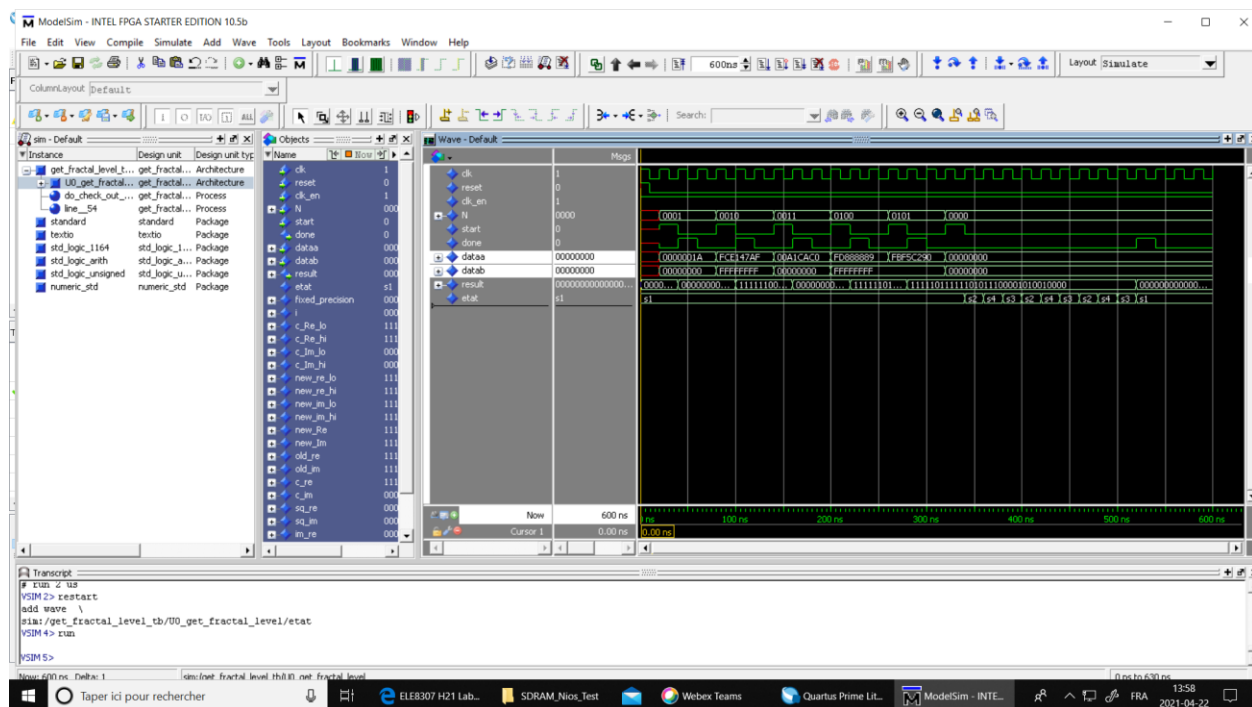


Figure 5 : Simulation de l'asm `get_fractal_level()`

Comme on peut le voir sur la figure précédente, l'asm retourne bien le bon nombre d'itération après qu'on lui a passé les bonnes valeurs en paramètres. Il est à noter que dans cette simulation l'état init est appelé S1, l'état S1 est appelé S2, l'état S2 est appelé S3 et l'état S3 est appelé S4. On peut aussi voir en outre que le signal `done` passe à un, neuf cycles d'horloge après l'appel à l'instruction de `N = 0`.

B. ASM2 pour set_pixel_ASM() (Michael Flandez)

Ici est présenté l'ASM actualisé avec les modifications nécessaires pour le faire travailler correctement.

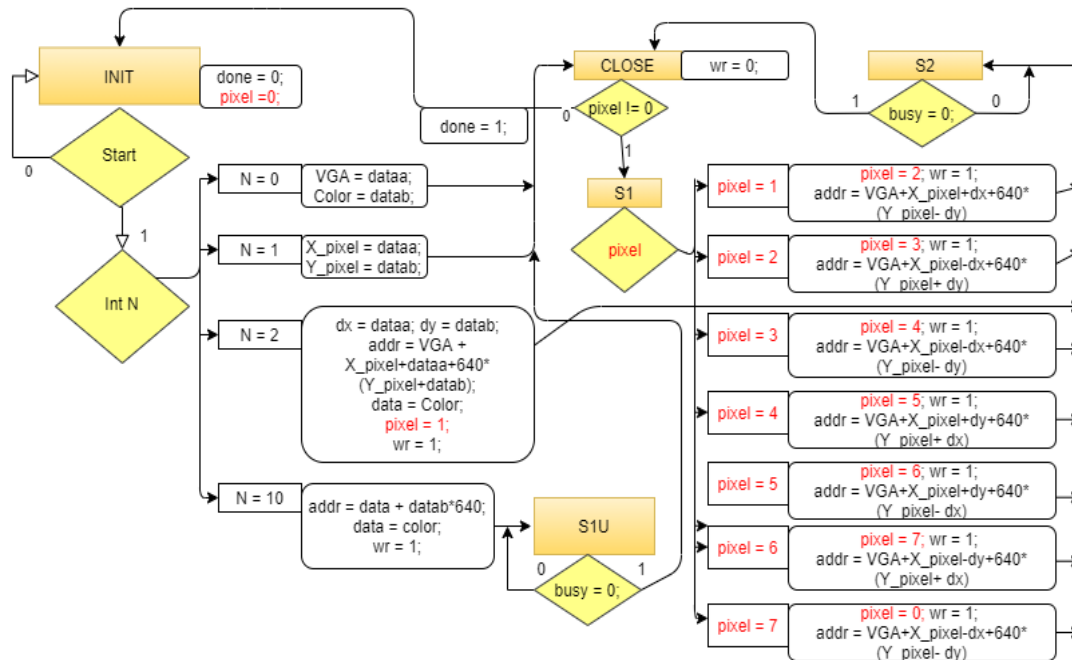


Figure 6: Schéma pour l'ASM set_pixel_ASM(). **Attention:** les calculs dans les bulles ont des multiplications, mais en vérité ce sont des décalages logiques à gauche. On a laissé les multiplications pour assouplir la lecture.

On voit au total cinq états. Le premier état **INIT** est le point de départ avec **pixel** mis à zéro et reste là jusqu'à avoir la commande **start** et 0, 1, 2 ou 10 en **N**. **Pixel** est le compteur pour calculer et dessiner huit pixels en séquence.

Ensuite, si **N** est zéro, les deux entrées **dataa** et **datab** sont stockés dans les variables internes **VGA** et **Color**. Au coup d'horloge suivant on passe à **CLOSE**. Là on vérifie la valeur de **pixel**, si **pixel** est égale à zéro alors on met **done** à un et on retourne à l'état **INIT**.

Donc là, si **start** le permet et **N** est égale à 1, donc on stocke **dataa** dans la variable **X_pixel** et **datab** dans la variable **Y_pixel**. Ensuite on arrive à **CLOSE** et étant donné que **pixel** est zéro on met **done** à un et on retourne à **INIT**.

Rapport – Fractale

Si start le permet et N est 2, on stock dataa dans **dx** et datab dans **dy** on met la valeur de pixel à un. On calcule également le première adresse en suivant l'algorithme des décalages logiques à gauche comme dans la formule 1 au point 3.1.B. et on l'assigne à la variable **addr**. Ce calcul d'adresse est écrit sur le bus Avalon directement dans le module **SDRAM**, qui affectera ensuite VGA. Par moyen du signal data, on envoie la valeur du signal Color et on mets **wr** à un, pour indiquer au processeur qu'on est en train d'écrire sur le bus Avalon. Au prochain coup d'horloge, on rentre dans l'état **S2** où on vérifie la condition de **busy**, si busy est égale à 1 on reste dans **S2** jusqu'à ce que busy devienne un, et là on passe à l'état **CLOSE**. Ici on met wr à zéro et on vérifie la condition si pixel est égale à zéro. Cette fois-ci, pixel n'est pas zéro donc on revient dans l'état **S1**. En S1 on vérifie la valeur de pixel et on calcule et assigne l'adresse suivante nouvellement à addr, on assigne la Color à data, on met wr à un, on modifie pixel et on le met à 2, et on passe au pixel suivant. On continue dans cette suite d'instructions jusqu'à pixel 7, endroit où on met la valeur de pixel à 0 de façon à pouvoir retourner à INIT and mettant done à 1, finalisant ainsi cette instruction multicyle d'écriture.

Pour ce qui est l'instruction N égale à 10, on calcule directement l'adresse à assigner à addr, on assigne Color à data, on met wr à 1, et on passe à l'état **S1U** au prochain coup d'horloge. Là on vérifie si busy est à zéro ou à un pour savoir si on doit attendre encore pour terminer l'écriture ou si on peut continuer à **CLOSE** et ainsi passer à **INIT** en mettant done à 1.

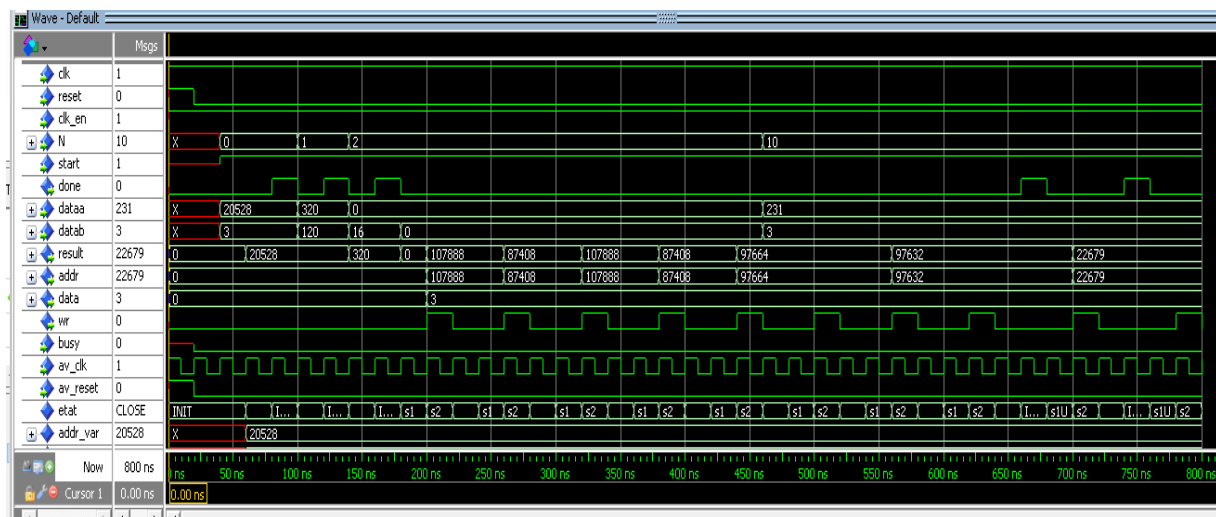


Figure 7: Simulation sur **ModelSIM** de l'ASM `set_pixel_ASM()`.

Cette figure permet de visualiser le comportement en simulation de ce ASM. La quatrième ligne correspond à N qui est le numéro d'instruction. Juste après 50 ns, N devint zéro, on stock dataa et datab. Si on regarde la sixième ligne, on peut voir que quelques cycles après done est mis à 1, lorsque l'instruction revient à **INIT**. Ensuite (done maintenant mis à zéro), N devient 1, dataa et datab sont stockés et done est mis à 1 encore une fois en revenant à **INIT** où done est remis à zéro. L'instruction est 2 et là on peut voir le signal wr sur la ligne 12. De ce moment, on voit une suite de 8 fois où le signal wr est mis à 1 et ensuite à 0. Après le huitième on voit que done est mis à 1, finissant ainsi l'instruction. Ensuite, l'instruction 10 est déclenché, suivie de done mis à 1 deux coups d'horloge après.

3.3 Les instructions spécialisées et les périphériques Avalon développés

C. ASM 1 pour get_fractal_level() (Sekou Kone)

Dans cette instruction, il s'agissait d'implémenter une instruction spécialisée multicycle pour effectuer la tâche de la fonction get_fractal_level(). Pour ce faire le problème du passage des valeurs paramètres c'est posé, nous avons donc dû réserver cinq cycles d'horloge pour transmettre :

- La valeur de fixed_precision (32 bits à travers dataa)
- La valeur de la constante réelle (64 bits à travers dataa et datab)
- La valeur de la constante imaginaire (64 bits à travers dataa et datab)
- La valeur de départ réelle (64 bits à travers dataa et datab)
- La valeur de départ imaginaire (64 bits à travers dataa et datab)

D. ASM2 pour set_pixel_ASM() (Michael Flandez)

Dans ce cas-ci, c'est une instruction avec écriture sur le bus Avalon qui est fait. On assigne trois valeurs pour faire l'écriture, une à addr (l'adresse dans l'espace mémoire dans la SDRAM), une autre à data (la valeur à être stocké à l'adresse mémoire addr) et une autre wr (indique au processeur qu'on veut écrire sur la SDRAM). Toujours à la fin d'instruction on doit mettre done à 1 pour indiquer que l'instruction a été terminée.

4 Résultats

- A. Les résultats de l'implémentation de ces ASM et de toutes les optimisations en logiciel sont résumés dans le tableau suivant.

fractal avec trois itérations	Avant	Après		
altera_avalon_jtag_uart_close	0	3,75 secondes		
draw_fractal()	4,97	2,07 secondes		
_muldi3	3,98	1,38 secondes	parent: draw_fractal()	
_udivsi3	11,35	0 secondes		
_umodsi3	9,9	0 secondes		
_muldf3	7,96	0 secondes		
_divdf3	5,41	0 secondes		
total	55,13	7,86 secondes	sans optimisation	

Figure 8: Tableau comparatif avant/après de l'implémentation des ASM et optimisations logicielles.

Dans la première colonne on nomme des instructions, dans la deuxième on a leurs durées de temps en secondes avant l'implémentation tandis qu'on leurs durées de temps après sont dans la troisième. Dans la cinquième, on a la fonction parent. À la partie inférieure, on a les valeurs totales pour une itération sans optimisation.

Les trois premières instructions sont les trois instructions les plus gourmandes en ressources après l'implémentation des optimisations et des ASM. On voit que altera_avalon_jtag_uart_close prend 3.75 secondes. Draw_fractal() a été réduite de 4.97 secondes à 2.07 secondes et _muldi3 a baissé de 3.98 secondes à 1.38 seconde.

Rapport – Fractale

Les quatre instructions qui suivent sont les instructions qui consommaient le plus de ressources avant l'implémentation. Là on voit des améliorations. Par exemple, `_divsi3` est passé de 11.35 secondes à 0 seconde, `_umodsi3` est passé de 9.9 secondes à 0 seconde, `_muldf3` est passé de 7.96 secondes à 0 seconde et `_divdf3` est passé de 5.41 secondes à 0 seconde.

Finalement, le total est passé de 55.13 secondes à 7.86 secondes. Il faut remarquer que les valeurs mesurées ici sont **sans optimisations**.

B. On a aussi mesuré la vitesse d'opération prenant en compte le temps que ça prend pour faire la suite d'opérations `update_position()`, `draw_factal()` et `draw_balls()`. Ceci a été fait en mode **run** avec des optimisations en niveau 3.

Comme résultats généraux on a obtenu:

situation	nbr de secondes	frames/s
début du jeu	1,6	0,63
zoom sur région très claire	1,74	0,57
gosse perle	1,91	0,52
grosse perle dans une région très claire	2,19	0,46

Figure 9: Résultats généraux en secondes et frames par seconde

Cette figure a trois colonnes, la première indiquant le contexte dans lequel le programme se trouvait sur l'écran, la deuxième avec le nombre de secondes que prenait chaque frame à charger et la troisième colonne avec le nombre de frames chargés sur l'écran par seconde.

5 Travaux subséquents à faire

Il est présent toujours la motivation pour améliorer le travail fait. On aimerait toujours accélérer plus l'application. Comme possibles candidats à implémenter, on propose les suivants:

- Création d'une instruction spécialisée multiplication afin d'accélérer les multiplications restantes dans les boucles **for**. Par exemple, après avoir fait un dernier profilage, on voit que la fonction **draw_fractal()** utilise 2.07 secondes en ressources en tandem avec `_muldi3` qu'utilise 1.38 seconde qui est-elle appelé aussi dans **draw_fractal()**.
- Il serait intéressant d'examiner un plus profondément les raisons pour avoir des temps plus lents dans les différentes situations décrites sur la figure 9.
- On aimerait aussi avoir l'opportunité de travailler pour améliorer la précision des données. En ce moment, on a réussi à augmenter la précision jusqu'à avoir 30 bits de précision en point fixe.

6 Conclusion

En somme, au cours de ce projet portant sur le prototypage d'une application produisant des fractales, il était question pour nous de trouver les moyens d'accélérer l'application. Pendant cette tâche, nous avons réalisé de nombreux apprentissages tels que la capacité de transformer efficacement le code c en version ASM, ou encore l'optimisation du nombre d'états d'une machine algorithmique en respectant les contraintes de timing. Dans un aspect plus pratique, nous avons appris comment paramétrer sous Quartus 18.1 les interfaces « custom instruction slave » et les interfaces pour bus Avalon.

Au cours de la réalisation du projet, nous avons rencontré des problèmes qui nous ont fait perdre inutilement du temps comme le fait qu'à un certain moment notre projet refusait de compiler et nous renvoyait l'erreur d'id 10228 : Verilog HDL error at de10_standard_qsys_sysid.v(34) : module « DE10_standard_qsys_sysid » cannot be declared more than once. Cette erreur venait tout simplement du fichier top level qui avait accidentellement été retiré du projet. Une autre erreur qui nous a fait perdre du temps est survenue lors de l'intégration de l'asm devant remplacer la fonction `get_fractal_level()`. D'abord elle nous retournait tout le temps 0. Cela était dû à la mauvaise configuration qui avait été faite du module « custom instruction slave ». L'horloge et le reset étaient externes au module ce qui faisait qu'il était considéré comme une instruction combinatoire et non pas comme une instruction multicycles. Par la suite, toujours avec l'asm de la fonction `get_fractal_level()`, nous avions un problème de valeurs de retours qui étaient incohérentes. Ce problème quant à lui était causé par un souci de timing en effet, les opérations dans certains états étaient gourmandes en temps et elles ne pouvaient donc s'exécuter en un seul cycle d'horloge. Ce problème a été résolu en introduisant des états intermédiaires pour alléger les calculs faits dans chaque état.

Maintenant, dans l'optique d'améliorer le cours et le projet nous suggérons de fournir aux étudiants plus de périodes où ils peuvent se rendre en laboratoire pour faire la mise en œuvre de leur code. Comme par le passé avant la pandémie où les étudiants pouvaient se rendre en permanence dans le local de projet pour y travailler.

Oui, nous autorisons le professeur à réutiliser notre travail.

A.1 Annexe

1. <https://stackoverflow.com/questions/922599/is-it-still-worth-trying-to-create-optimizations-for-sqrt-in-c> Resource trouvé qu'optimise et accélère le calcul de la racine carrée en utilisant une interpolation.